

Architecture Decision Record (ADR)

Christmas Tracker iOS Application

Version: 1.0
Date: February 3, 2026
Status: Approved
Author: Development Team

Executive Summary

This document outlines the architectural decisions for the Christmas Tracker iOS application, a native Swift application designed to help families coordinate holiday gift-giving while preserving surprises. The application follows modern iOS development best practices using Swift 6, SwiftUI, and a service-oriented architecture with domain-partitioned data stores.

Key Architectural Principles:

- **Swift 6 First:** Strict concurrency checking, structured concurrency, all actors are Sendable
- **Feature-Driven:** Focus on core functionality over polish, iterate based on user feedback
- **Testable:** Protocol-based design with mock implementations, 80% unit test coverage target
- **Observable:** Comprehensive logging and observability from day one
- **Scalable:** Clean separation of concerns, easy to extend with new features

1. Platform Requirements

1.1 Target Platform

Platform: iOS 26.0+
Swift Version: 6.0
Xcode: 26.0.1 or later
Minimum Device: iPhone 11 (A13 Bionic or newer)

Rationale:

- **iOS 26.0:** Current stable release (September 2025), aligns with Apple's unified versioning across all platforms
- **Swift 6:** Enables strict concurrency checking, eliminates data races at compile time, modern async/await patterns
- **iPhone 11+:** A13 Bionic minimum requirement for iOS 26, provides good device coverage while enabling modern features

Supported Devices:

- iPhone 11, 11 Pro, 11 Pro Max
- iPhone 12, 12 mini, 12 Pro, 12 Pro Max
- iPhone 13, 13 mini, 13 Pro, 13 Pro Max
- iPhone 14, 14 Plus, 14 Pro, 14 Pro Max
- iPhone 15, 15 Plus, 15 Pro, 15 Pro Max
- iPhone SE (2nd generation and later)

Dropped Devices: iPhone XS, XS Max, XR (iOS 26 requirement)

1.2 Swift 6 Concurrency Requirements

```
// Project Build Settings
SWIFT_VERSION = 6.0
```

```
SWIFT_STRICT_CONCURRENCY = complete
```

```
// This enforces:  
// ✓ All actors must be Sendable  
// ✓ All @MainActor usage must be explicit  
// ✓ Data races caught at compile time  
// ✓ Structured concurrency required
```

Impact on Architecture:

- All Services are `@MainActor @Observable` (UI updates on main thread)
- All Repositories and DataStores are `actor` (background thread-safe operations)
- All Models are `Sendable` structs (safe to pass across concurrency boundaries)
- No `@unchecked Sendable` allowed (maintain strict type safety)

2. Architectural Patterns

2.1 Service-Oriented Architecture with Observation Framework

We use a service-oriented architecture where services manage business logic and coordinate between data stores and the UI.

```
// Services are @MainActor and @Observable for SwiftUI integration  
@MainActor  
@Observable  
class ListService {  
    private let repository: ListRepository  
    private let datastore: ListDataStore  
  
    var lists: [GiftList] = []  
    var isLoading: Bool = false  
    var error: Error?  
  
    init(repository: ListRepository, datastore: ListDataStore) {  
        self.repository = repository  
        self.dataStore = datastore  
    }  
  
    func fetchLists() async {  
        isLoading = true  
        defer { isLoading = false }  
  
        do {  
            // Fetch from repository (handles caching)  
            let lists = try await repository.fetchLists()  
  
            // Update data store  
            await datastore.setLists(lists)  
  
            // Update local state (triggers UI refresh)  
            self.lists = lists  
        } catch {  
            self.error = error  
            await Logger.shared.log(.error, "Failed to fetch lists: \(error)")  
        }  
    }  
}
```

Rationale:

- **@MainActor:** Ensures all service operations happen on main thread, safe for SwiftUI
- **@Observable:** SwiftUI automatically observes changes, re-renders views efficiently
- **No Redux boilerplate:** Simpler than Redux while maintaining predictable state flow
- **Clear ownership:** Each service owns its domain, no "god objects"

2.2 Domain-Partitioned Data Stores

Data stores are actors that manage in-memory caches of domain data, providing thread-safe access.

```
// Data stores are actors for thread-safe background operations
actor ListDataStore {
    private var lists: [String: GiftList] = [:]
    private var listStats: [String: ListStats] = [:]

    func setLists(_ lists: [GiftList]) {
        for list in lists {
            self.lists[list.id] = list
        }
    }

    func getList(_ id: String) -> GiftList? {
        lists[id]
    }

    func updateListStats(_ stats: ListStats, for listId: String) {
        listStats[listId] = stats
    }

    func clear() {
        lists.removeAll()
        listStats.removeAll()
    }
}

actor ItemDataStore {
    private var items: [String: GiftItem] = [:]

    func setItems(_ items: [GiftItem]) {
        for item in items {
            self.items[item.id] = item
        }
    }

    func updateItem(_ item: GiftItem) {
        items[item.id] = item
    }

    func clear() {
        items.removeAll()
    }
}

actor UserDataStore {
    private var users: [String: User] = [:]
    private var currentUser: User?

    func setCurrentUser(_ user: User) {
        currentUser = user
        users[user.id] = user
    }
}
```

```

func getCurrentUser() -> User? {
    currentUser
}

func clear() {
    users.removeAll()
    currentUser = nil
}
}

```

Rationale:

- **Domain partitioning:** Prevents "god object" anti-pattern, each store owns its data
- **Actor isolation:** Thread-safe by default, no manual locking required
- **In-memory cache:** Fast access, cleared on logout/session expiry
- **Coordinated updates:** Services orchestrate multi-store updates when needed

Cross-Store Coordination Example:

```

@MainActor
@Observable
class ItemService {
    private let itemRepository: ItemRepository
    private let itemStore: ItemDataStore
    private let listStore: ListDataStore

    func purchaseItem(_ itemId: String) async throws {
        // Update item
        let updatedItem = try await itemRepository.purchaseItem(itemId)
        await itemStore.updateItem(updatedItem)

        // Refresh parent list to update stats
        if let list = await listStore.getList(updatedItem.listId) {
            let updatedList = try await listRepository.fetchList(updatedItem.listId)
            await listStore.setLists([updatedList])

            // Update UI state
            self.items = self.items.map { $0.id == itemId ? updatedItem : $0 }
        }
    }
}

```

2.3 Repository Pattern

Repositories abstract data access, providing a protocol-based interface that can be swapped between network and mock implementations.

```

// Protocol defining data access contract
protocol ListRepository: Sendable {
    func fetchLists() async throws -> [GiftList]
    func fetchList(_ id: String) async throws -> GiftList
    func createList(_ list: CreateListRequest) async throws -> GiftList
    func updateList(_ id: String, _ list: UpdateListRequest) async throws -> GiftList
    func deleteList(_ id: String) async throws
}

// Production implementation with caching
actor NetworkListRepository: ListRepository {
    private let apiClient: APIClient
}

```

```

private let cache: NetworkCache

init(apiClient: APIClient, cache: NetworkCache) {
    self.apiClient = apiClient
    self.cache = cache
}

func fetchLists() async throws -> [GiftList] {
    // Check cache first
    if let cached: [GiftList] = await cache.get("lists") {
        return cached
    }

    // Fetch from network
    let lists: [GiftList] = try await apiClient.request(
        .get,
        path: "/lists"
    )

    // Update cache
    await cache.set("lists", value: lists)

    return lists
}

// ... other methods
}

// Mock implementation for testing/previews
actor MockListRepository: ListRepository {
    var mockLists: [GiftList] = []
    var shouldThrowError: Error?

    func fetchLists() async throws -> [GiftList] {
        if let error = shouldThrowError {
            throw error
        }
        return mockLists
    }

    // ... other methods
}

```

Rationale:

- **Protocol-based:** Enables dependency injection, easy to swap implementations
- **Testability:** Mock repositories for unit tests, no network calls required
- **SwiftUI Previews:** Use mock data for fast preview rendering
- **Caching:** Network repository handles caching transparently
- **Actor isolation:** Thread-safe network operations

2.4 Dependency Injection via Environment

Services and repositories are injected via SwiftUI's Environment system.

```

// Define environment keys
private struct ListServiceKey: EnvironmentKey {
    static let defaultValue: ListService = ListService(
        repository: NetworkListRepository(
            apiClient: .shared,

```

```

        cache: .shared
    ),
    datastore: ListDataStore()
)
}

extension EnvironmentValues {
    var listService: ListService {
        get { self[ListServiceKey.self] }
        set { self[ListServiceKey.self] = newValue }
    }
}

// Usage in views
struct DashboardView: View {
    @Environment(\.listService) private var listService

    var body: some View {
        List(listService.lists) { list in
            ListCard(list: list)
        }
        .task {
            await listService.fetchLists()
        }
    }
}

// Override for previews
#Preview {
    DashboardView()
        .environment(\.listService, ListService(
            repository: MockListRepository(),
            datastore: ListDataStore()
        ))
}

```

Rationale:

- **No singletons:** Avoids global mutable state, better testability
- **Easy overrides:** Simple to inject mocks for previews and tests
- **SwiftUI native:** Leverages framework's dependency injection system
- **Type-safe:** Compiler enforces correct types

3. Configuration Management

3.1 xcconfig-Based Configuration

Configuration values are stored in xcconfig files, separate for production and debug builds.

```

# Config.xcconfig (Production)
IOS_DEPLOYMENT_TARGET = 26.0
SWIFT_VERSION = 6.0
CACHE_DURATION = 900           // 15 minutes
SESSION_TIMEOUT = 7200         // 2 hours
BACKGROUND_REFRESH = 900       // 15 minutes
LOG_FLUSH_INTERVAL = 60        // 60 seconds

# Config-Debug.xcconfig
IOS_DEPLOYMENT_TARGET = 26.0

```

```

SWIFT_VERSION = 6.0
CACHE_DURATION = 30           // 30 seconds (faster testing)
SESSION_TIMEOUT = 300         // 5 minutes
BACKGROUND_REFRESH = 60       // 1 minute
LOG_FLUSH_INTERVAL = 15       // 15 seconds

```

```

// AppConfiguration.swift reads from xcconfig
struct AppConfiguration {
    static let cacheDuration: TimeInterval = {
        guard let value = Bundle.main.infoDictionary?["CACHE_DURATION"] as? String,
            let duration = TimeInterval(value) else {
            return 900 // Default 15 minutes
        }
        return duration
    }()

    static let sessionTimeout: TimeInterval = {
        guard let value = Bundle.main.infoDictionary?["SESSION_TIMEOUT"] as? String,
            let duration = TimeInterval(value) else {
            return 7200 // Default 2 hours
        }
        return duration
    }()

    static let backgroundRefreshThreshold: TimeInterval = {
        guard let value = Bundle.main.infoDictionary?["BACKGROUND_REFRESH"] as? String,
            let duration = TimeInterval(value) else {
            return 900 // Default 15 minutes
        }
        return duration
    }()

    static let logFlushInterval: TimeInterval = {
        guard let value = Bundle.main.infoDictionary?["LOG_FLUSH_INTERVAL"] as? String,
            let duration = TimeInterval(value) else {
            return 60 // Default 60 seconds
        }
        return duration
    }()

    static let environment: String = {
        #if DEBUG
        return "debug"
        #else
        return "production"
        #endif
    }()
}

```

Rationale:

- **Build-time configuration:** No runtime switches, faster execution
- **Easy to modify:** Change timeouts without code changes
- **Scheme-based:** Different configs for Debug vs Release schemes
- **Version controlled:** xcconfig files committed to repo

4. Networking & Caching

4.1 Actor-Based API Client

The API client is implemented as an actor for thread-safe network operations.

```
actor APIClient {
    static let shared = APIClient()

    private let baseURL: URL
    private let session: URLSession
    private let cache: NetworkCache

    init(session: URLSession = .shared, cache: NetworkCache = .shared) {
        // Load base URL from xcconfig
        guard let urlString = Bundle.main.infoDictionary?["API_BASE_URL"] as? String,
              let url = URL(string: urlString) else {
            fatalError("API_BASE_URL not configured in xcconfig")
        }
        self.baseURL = url
        self.session = session
        self.cache = cache
    }

    init(session: URLSession = .shared, cache: NetworkCache = .shared) {
        self.session = session
        self.cache = cache
    }

    func request<T: Decodable>(
        _ method: HTTPMethod,
        path: String,
        body: (any Encodable)? = nil,
        forceRefresh: Bool = false,
        cacheScope: CacheScope = .auto
    ) async throws -> T {
        // Build request
        var request = URLRequest(url: baseURL.appendingPathComponent(path))
        request.httpMethod = method.rawValue
        request.setValue("application/json", forHTTPHeaderField: "Content-Type")

        // Add auth token if available
        if let token = await AuthenticationService.shared.accessToken {
            request.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization")
        }

        // Add body if present
        if let body = body {
            request.httpBody = try JSONEncoder().encode(body)
        }

        // Check cache (skip if forceRefresh or not GET)
        if method == .get && !forceRefresh {
            if let cached: T = await cache.get(path) {
                await Logger.shared.log(.debug, "Cache hit: \(path)", category: .network)
                return cached
            }
        }

        // Perform request
        await Logger.shared.log(.debug, "\(method.rawValue) \(path)", category: .network)
        let startTime = Date()

        let (data, response) = try await session.data(for: request)
```

```

let duration = Date().timeIntervalSince(startTime)

guard let httpResponse = response as? HTTPURLResponse else {
    throw APIError.invalidResponse
}

await Logger.shared.log(
    .info,
    "\\(method.rawValue) \\(path) -> \\(httpResponse.statusCode) \\(String(format: "%.2f",
duration))s)",
    category: .network
)

// Handle errors
guard (200...299).contains(httpResponse.statusCode) else {
    if httpResponse.statusCode == 401 {
        // Session expired, trigger global logout
        await ErrorInterceptor.shared.handle401()
    }
    throw APIError.httpError(statusCode: httpResponse.statusCode, data: data)
}

// Decode response
let decoded = try JSONDecoder().decode(T.self, from: data)

// Cache if GET request
if method == .get {
    await cache.set(path, value: decoded)
}

// Handle cache eviction based on method and scope
if method != .get {
    await handleCacheEviction(scope: cacheScope, path: path)
}

return decoded
}

private func handleCacheEviction(scope: CacheScope, path: String) async {
    switch scope {
    case .none:
        break // No cache eviction
    case .lists:
        await cache.invalidate(scope: .lists)
    case .items:
        await cache.invalidate(scope: .items)
    case .all:
        await cache.invalidateAll()
    case .auto:
        // Auto-detect based on path
        if path.contains("/lists") {
            await cache.invalidate(scope: .lists)
        } else if path.contains("/items") {
            await cache.invalidate(scope: .items)
        }
    }
}

}

/// Cache scope for selective eviction
enum CacheScope: Sendable {
    case none // No eviction (profile updates, settings)

```

```

    case lists      // Evict list-related caches
    case items     // Evict item-related caches
    case all       // Evict all caches
    case auto      // Auto-detect from path
}

// Usage examples in repositories:

// Profile update - no cache eviction
let user: User = try await apiClient.request(
    .put,
    path: "/profile",
    body: updateRequest,
    cacheScope: .none
)

// Purchase item - evict both items and lists
let item: GiftItem = try await apiClient.request(
    .post,
    path: "/items/\(itemId)/purchase",
    cacheScope: .all // Lists need refresh for stats
)

enum HTTPMethod: String {
    case get = "GET"
    case post = "POST"
    case put = "PUT"
    case delete = "DELETE"
    case patch = "PATCH"
}

enum APIError: Error {
    case invalidResponse
    case httpError(statusCode: Int, data: Data)
    case decodingError(Error)
}

```

4.2 Session-Scoped Network Cache

The cache is session-scoped and automatically cleared on logout or 401 errors.

```

actor NetworkCache {
    static let shared = NetworkCache()

    struct CachedResponse<T> {
        let value: T
        let timestamp: Date
        let sessionId: String
    }

    private var cache: [String: Any] = [:]
    private var currentSessionId: String?

    func setSessionId(_ id: String) {
        if currentSessionId != id {
            // Session changed, clear cache
            cache.removeAll()
            currentSessionId = id
        }
    }
}

```

```

func get<T>(_ key: String) -> T? {
    guard let cached = cache[key] as? CachedResponse<T>,
          cached.sessionId == currentSessionId,
          Date().timeIntervalSince(cached.timestamp) < AppConfig.cacheDuration
    else {
        return nil
    }
    return cached.value
}

func set<T>(_ key: String, value: T) {
    guard let sessionId = currentSessionId else { return }
    cache[key] = CachedResponse(
        value: value,
        timestamp: Date(),
        sessionId: sessionId
    )
}

func invalidate(_ key: String) {
    cache.removeValue(forKey: key)
}

func invalidate(scope: CacheScope) {
    switch scope {
    case .lists:
        cache = cache.filter { !$0.key.contains("/lists") }
    case .items:
        cache = cache.filter { !$0.key.contains("/items") }
    case .all, .auto, .none:
        break // Handled elsewhere
    }
}

func invalidateAll() {
    cache.removeAll()
}

func clear() {
    cache.removeAll()
    currentSessionId = nil
}
}

```

Caching Rules:

1. **Cache duration:** 15 minutes (production), 30 seconds (debug)
2. **Cache scope:** Session-only, cleared on logout or 401
3. **Cache invalidation:**
 - All write operations (POST/PUT/DELETE) clear entire cache
 - Pull-to-refresh uses `forceRefresh: true` to bypass cache
 - Background refresh (>15 min) bypasses cache
4. **Stale-while-revalidate:** Show cached data immediately, fetch fresh in background

5. Authentication & Security

5.1 Device Fingerprint

Device fingerprints uniquely identify devices for token validation.

```

actor DeviceFingerprintService {
    static let shared = DeviceFingerprintService()

    private let keychainKey = "com.christmastracker.deviceFingerprintUUID"

    func getFingerprint() async throws -> String {
        // Get or create base UUID
        let uuid = try await getOrCreateUUID()

        // Append device info dynamically
        let model = UIDevice.current.model // "iPhone"
        let version = UIDevice.current.systemVersion // "26.0"

        return "\(uuid)-\(model)-\(version)"
    }

    private func getOrCreateUUID() async throws -> String {
        // Try to retrieve from Keychain
        if let existing = try? KeychainService.shared.get(keychainKey) {
            return existing
        }

        // Generate new UUID
        let uuid = UUID().uuidString

        // Store in Keychain (persists across reinstalls)
        try KeychainService.shared.set(uuid, for: keychainKey, accessControl: .afterFirstUnlock)

        return uuid
    }
}

```

Format: "550e8400-e29b-41d4-a716-446655440000-iPhone-26.0"

Properties:

- **Persists across reinstalls:** UUID stored in Keychain with `.afterFirstUnlock`
- **Changes on OS updates:** Version component updates automatically
- **Debuggable:** Human-readable format, not hashed
- **Privacy-preserving:** No personally identifiable information

5.2 Authentication Flows

Password Authentication (Web + iOS)

```

// POST /auth/login
struct LoginRequest: Codable, Sendable {
    let email: String
    let password: String
    let deviceFingerprint: String
}

struct LoginResponse: Codable, Sendable {
    let accessToken: String
    let refreshToken: String
    let user: User
}

@MainActor

```

```

@Observable
class AuthenticationService {
    private let repository: AuthRepository
    private let keychainService: KeychainService
    private let fingerprintService: DeviceFingerprintService

    var currentUser: User?
    var isAuthenticated: Bool { currentUser != nil }

    func login(email: String, password: String) async throws {
        let fingerprint = try await fingerprintService.getFingerprint()

        let request = LoginRequest(
            email: email,
            password: password,
            deviceFingerprint: fingerprint
        )

        let response: LoginResponse = try await repository.login(request)

        // Store tokens in Keychain
        try keychainService.set(response.accessToken, for: "accessToken")
        try keychainService.set(response.refreshToken, for: "refreshToken", accessControl:
.biometry)

        // Update session
        currentUser = response.user
        await NetworkCache.shared.setSessionId(response.user.id)

        // Track last password login
        UserDefaults.standard.set(Date(), forKey: "lastPasswordLogin")

        await Logger.shared.log(.info, "User logged in: \(response.user.email)", category: .auth)
    }
}

```

Biometric Authentication (iOS Only)

```

// POST /auth/biometric
struct BiometricAuthRequest: Codable, Sendable {
    let refreshToken: String
    let deviceFingerprint: String
}

extension AuthenticationService {
    func authenticateWithBiometrics() async throws {
        // Check if biometric is available
        let context = LAContext()
        var error: NSError?
        guard context.canEvaluatePolicy(.deviceOwnerAuthenticationWithBiometrics, error: &error)
    else {
        throw AuthError.biometricNotAvailable
    }

    // Prompt for biometric
    guard try await context.evaluatePolicy(
        .deviceOwnerAuthenticationWithBiometrics,
        localizedReason: "Authenticate to access Christmas Tracker"
    ) else {
        throw AuthError.biometricFailed
    }
}

```

```

    }

    // Retrieve refresh token from Keychain (behind biometric)
    guard let refreshToken = try? keychainService.get("refreshToken") else {
        throw AuthError.noRefreshToken
    }

    let fingerprint = try await fingerprintService.getFingerprint()

    let request = BiometricAuthRequest(
        refreshToken: refreshToken,
        deviceFingerprint: fingerprint
    )

    let response: LoginResponse = try await repository.biometricAuth(request)

    // Store new tokens (rotated)
    try keychainService.set(response.accessToken, for: "accessToken")
    try keychainService.set(response.refreshToken, for: "refreshToken", accessControl:
.biometry)

    // Update session
    currentUser = response.user
    await NetworkCache.shared.setSessionId(response.user.id)

    await Logger.shared.log(.info, "User authenticated with biometrics", category: .auth)
}
}

```

Token Refresh

```

// POST /auth/refresh
struct RefreshRequest: Codable, Sendable {
    let refreshToken: String
    let deviceFingerprint: String
}

extension AuthenticationService {
    func refreshAccessToken() async throws {
        guard let refreshToken = try? keychainService.get("refreshToken") else {
            throw AuthError.noRefreshToken
        }

        let fingerprint = try await fingerprintService.getFingerprint()

        let request = RefreshRequest(
            refreshToken: refreshToken,
            deviceFingerprint: fingerprint
        )

        let response: LoginResponse = try await repository.refresh(request)

        // Store new tokens (rotated)
        try keychainService.set(response.accessToken, for: "accessToken")
        try keychainService.set(response.refreshToken, for: "refreshToken", accessControl:
.biometry)

        await Logger.shared.log(.info, "Access token refreshed", category: .auth)
    }
}

```

5.3 Token Management

Token Lifecycle:

1. **Access Token:** Short-lived (2 hours), stored in Keychain (no biometric)
2. **Refresh Token:** Long-lived (30 days), stored in Keychain behind biometric, rotated on each use
3. **Device Fingerprint:** Associated with refresh token on backend, validated on each use

Token Rotation:

- Both `/auth/biometric` and `/auth/refresh` endpoints return NEW refresh tokens
- Old refresh tokens are immediately invalidated on backend
- Prevents token reuse attacks

Per-Device Tokens:

- Each device gets its own refresh token
- Maximum 10 devices per user
- Logging in on a device invalidates previous tokens for that device fingerprint

Re-Authentication:

- Backend tracks `lastPasswordLogin` timestamp
- After 30 days, refresh token expires
- User must log in with password to get new refresh token

5.4 Keep-Alive Session Management

To provide better UX and reduce re-authentication friction, the app implements activity-based session extension via periodic ping requests.

Design Principles:

- Base timeout of 30 minutes of inactivity
- Active users send ping every 5 minutes to extend session
- Maximum session lifetime of 24 hours (force re-auth after)
- Inactive sessions expire faster

```
actor SessionManager {
    static let shared = SessionManager()

    private var lastActivityTimestamp: Date = .now
    private var sessionExpiryDate: Date
    private var sessionStartDate: Date = .now

    private let baseTimeout: TimeInterval = 30 * 60 // 30 minutes
    private let pingInterval: TimeInterval = 5 * 60 // 5 minutes
    private let maxSessionLifetime: TimeInterval = 24 * 60 * 60 // 24 hours

    private var pingTask: Task<Void, Never>?

    init() {
        sessionExpiryDate = Date().addingTimeInterval(baseTimeout)
        startPingTask()
    }

    private func startPingTask() {
        pingTask = Task.detached(priority: .utility) {
            while !Task.isCancelled {
                try? await Task.sleep(for: .seconds(await self.pingInterval))
            }
        }
    }
}
```

```

        await self.checkAndPing()
    }
}

private func checkAndPing() async {
    // Check if user has been active recently
    let timeSinceLastActivity = Date().timeIntervalSince(lastActivityTimestamp)
    let timeSinceSessionStart = Date().timeIntervalSince(sessionStartDate)

    // Don't ping if:
    // 1. User inactive for > ping interval
    // 2. Session exceeded max lifetime
    guard timeSinceLastActivity < pingInterval,
          timeSinceSessionStart < maxSessionLifetime else {
        return
    }

    // Send ping to extend session
    await sendPing()
}

private func sendPing() async {
    do {
        let response: PingResponse = try await APIClient.shared.request(
            .post,
            path: "/auth/ping",
            cacheScope: .none
        )

        // Update expiry based on server response
        sessionExpiryDate = response.expiresAt

        logDebug("Session extended via ping", category: .auth, metadata: [
            "expiresAt": response.expiresAt.ISO8601Format()
        ])
    } catch {
        logWarning("Ping failed: \(error)", category: .auth)
        // Don't force logout on ping failure (network might be temporarily down)
    }
}

func recordActivity() {
    lastActivityTimestamp = .now
}

func isSessionValid() -> Bool {
    let now = Date()
    let timeSinceStart = now.timeIntervalSince(sessionStartDate)

    // Session invalid if:
    // 1. Expired (no activity)
    // 2. Exceeded max lifetime
    return now < sessionExpiryDate && timeSinceStart < maxSessionLifetime
}

func startNewSession() {
    sessionStartDate = .now
    sessionExpiryDate = Date().addingTimeInterval(baseTimeout)
    lastActivityTimestamp = .now
}

func endSession() {

```

```

        pingTask?.cancel()
        pingTask = nil
    }
}

struct PingResponse: Codable, Sendable {
    let expiresAt: Date
    let ttl: Int // Time-to-live in seconds
}

// Track activity throughout app
extension View {
    func trackUserActivity() -> some View {
        self
            .onAppear {
                Task {
                    await SessionManager.shared.recordActivity()
                }
            }
            .gesture(
                TapGesture()
                    .onEnded { _ in
                        Task {
                            await SessionManager.shared.recordActivity()
                        }
                    }
            )
    }
}

// Usage in views
struct DashboardView: View {
    var body: some View {
        ScrollView {
            // Content
        }
        .trackUserActivity()
    }
}

```

Backend Endpoint Required:

```

POST /auth/ping
Headers: { Authorization: Bearer <accessToken> }
Response: {
    expiresAt: string, // ISO 8601 timestamp
    ttl: number // Seconds until expiry
}

```

Integration in AuthenticationService:

```

extension AuthenticationService {
    func login(email: String, password: String) async throws {
        // ... existing login logic ...

        // Start session manager
        await SessionManager.shared.startNewSession()
    }
}

```

```

func logout() async throws {
    // End session manager
    await SessionManager.shared.endSession()

    // ... existing logout logic ...
}
}

```

5.5 Keychain Service

```

actor KeychainService {
    static let shared = KeychainService()

    enum AccessControl {
        case afterFirstUnlock
        case biometry
    }

    func set(_ value: String, for key: String, accessControl: AccessControl = .afterFirstUnlock)
    throws {
        let data = value.data(using: .utf8)!

        var query: [String: Any] = [
            kSecClass as String: kSecClassGenericPassword,
            kSecAttrAccount as String: key,
            kSecValueData as String: data
        ]

        // Add access control if biometry
        if accessControl == .biometry {
            let access = SecAccessControlCreateWithFlags(
                nil,
                kSecAttrAccessibleWhenUnlockedThisDeviceOnly,
                .biometryCurrentSet,
                nil
            )!
            query[kSecAttrAccessControl as String] = access
        } else {
            query[kSecAttrAccessible as String] = kSecAttrAccessibleAfterFirstUnlock
        }

        // Delete existing item
        SecItemDelete(query as CFDictionary)

        // Add new item
        let status = SecItemAdd(query as CFDictionary, nil)
        guard status == errSecSuccess else {
            throw KeychainError.unableToStore
        }
    }

    func get(_ key: String) throws -> String {
        let query: [String: Any] = [
            kSecClass as String: kSecClassGenericPassword,
            kSecAttrAccount as String: key,
            kSecReturnData as String: true
        ]

        var result: AnyObject?
    }
}

```

```

        let status = SecItemCopyMatching(query as CFDictionary, &result)

        guard status == errSecSuccess,
            let data = result as? Data,
            let string = String(data: data, encoding: .utf8) else {
            throw KeychainError.itemNotFound
        }

        return string
    }

    func delete(_ key: String) throws {
        let query: [String: Any] = [
            kSecClass as String: kSecClassGenericPassword,
            kSecAttrAccount as String: key
        ]

        let status = SecItemDelete(query as CFDictionary)
        guard status == errSecSuccess || status == errSecItemNotFound else {
            throw KeychainError.unableToDelete
        }
    }

    func clear() throws {
        let query: [String: Any] = [
            kSecClass as String: kSecClassGenericPassword
        ]

        SecItemDelete(query as CFDictionary)
    }
}

enum KeychainError: Error {
    case unableToStore
    case itemNotFound
    case unableToDelete
}

```

6. Logging & Observability

6.1 Logging Architecture

Comprehensive logging system with multiple destinations, automatic PII filtering, and configurable flush intervals.

```

actor Logger {
    static let shared = Logger()

    private var destinations: [LogDestination] = []
    private var buffer: [LogEvent] = []

    init() {
        setupDestinations()
        startAutoFlush()
    }

    private func setupDestinations() {
        #if DEBUG
        destinations.append(OSLogDestination())
        destinations.append(ConsoleDestination())

```

```

// Try New Relic, fallback to backend
if let newRelic = NewRelicDestination() {
    destinations.append(newRelic)
} else {
    destinations.append(BackendDestination())
}
#else
destinations.append(OSLogDestination())

if let newRelic = NewRelicDestination() {
    destinations.append(newRelic)
} else {
    destinations.append(BackendDestination())
}
#endif
}

func log(
    _ level: LogLevel,
    _ message: String,
    category: LogCategory,
    metadata: [String: String] = [:],
    file: String = #file,
    function: String = #function,
    line: Int = #line
) async {
    // Filter PII
    let sanitizedMessage = await PIIFilter.shared.redact(message)
    let sanitizedMetadata = await withTaskGroup(of: (String, String).self) { group in
        for (key, value) in metadata {
            group.addTask {
                (key, await PIIFilter.shared.redact(value))
            }
        }

        var result: [String: String] = [:]
        for await (key, value) in group {
            result[key] = value
        }
        return result
    }

    let event = LogEvent(
        timestamp: Date(),
        level: level,
        category: category,
        message: sanitizedMessage,
        metadata: sanitizedMetadata.merging([
            "file": file.components(separatedBy: "/").last ?? file,
            "function": function,
            "line": "\(line)"
        ]) { $1 },
        signpost: nil,
        duration: nil,
        memoryUsage: nil
    )

    buffer.append(event)

    // Immediate flush for errors/critical
    if level == .error || level == .critical {
        await flush()
    }
}

```

```

    }
}

private func startAutoFlush() {
    Task {
        while !Task.isCancelled {
            try? await Task.sleep(for: .seconds(AppConfiguration.logFlushInterval))
            await flush()
        }
    }
}

func flush() async {
    guard !buffer.isEmpty else { return }

    let events = buffer
    buffer.removeAll()

    // Flush to all destinations in parallel
    await withTaskGroup(of: Void.self) { group in
        for destination in destinations {
            group.addTask {
                await destination.flush(events)
            }
        }
    }
}

// Called by AppDelegate
func flushOnBackground() async {
    await flush()
}

func flushOnForeground() async {
    await flush()
}

func flushOnLogout() async {
    await flush()
}
}

// Convenience functions (fire-and-forget, non-blocking)
func logDebug(_ message: String, category: LogCategory = .business, metadata: [String: String] = [:]) {
    Task.detached(priority: .background) {
        await Logger.shared.log(.debug, message, category: category, metadata: metadata)
    }
}

func logInfo(_ message: String, category: LogCategory = .business, metadata: [String: String] = [:]) {
    Task.detached(priority: .background) {
        await Logger.shared.log(.info, message, category: category, metadata: metadata)
    }
}

func logWarning(_ message: String, category: LogCategory = .business, metadata: [String: String] = [:]) {
    Task.detached(priority: .background) {
        await Logger.shared.log(.warning, message, category: category, metadata: metadata)
    }
}
}

```

```

func logError(_ message: String, category: LogCategory = .business, metadata: [String: String] = [:]) {
    Task.detached(priority: .background) {
        await Logger.shared.log(.error, message, category: category, metadata: metadata)
    }
}

func logCritical(_ message: String, category: LogCategory = .business, metadata: [String: String] = [:]) {
    Task.detached(priority: .background) {
        await Logger.shared.log(.critical, message, category: category, metadata: metadata)
    }
}

```

6.2 Log Event Model

```

struct LogEvent: Sendable, Codable {
    let timestamp: Date
    let level: LogLevel
    let category: LogCategory
    let message: String
    let metadata: [String: String]
    let signpost: SignpostInfo?
    let duration: TimeInterval?
    let memoryUsage: UInt64?
}

enum LogLevel: String, Codable, Sendable {
    case debug
    case info
    case warning
    case error
    case critical
}

enum LogCategory: String, Codable, Sendable {
    case network // API calls, network errors, latency
    case auth // Login, logout, token refresh, biometric
    case ui // View lifecycle, user interactions, navigation
    case cache // Cache hits/misses, invalidation
    case performance // Signposts, memory, slow operations
    case business // Feature events (purchase, invite, create list)
}

struct SignpostInfo: Sendable, Codable {
    let name: String
    let id: UUID
    let state: SignpostState
}

enum SignpostState: String, Codable, Sendable {
    case begin
    case end
    case event
}

```

6.3 PII Filtering

```

actor PIIFilter {
    static let shared = PIIFilter()

    private let patterns: [NSRegularExpression] = [
        try! NSRegularExpression(pattern: #"\\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Z|a-z]{2,}\\b"#),
// Email
        try! NSRegularExpression(pattern: #"\\bBearer\\s+[A-Za-z0-9\\-._~+/]+=*"#), // Bearer token
        try! NSRegularExpression(pattern: #"\\b[0-9]{3}-[0-9]{2}-[0-9]{4}\\b"#), // SSN
        try! NSRegularExpression(pattern: #"\\b[0-9]{16}\\b"#), // Credit card
        try! NSRegularExpression(pattern: #"\\b[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}\\b"#), // UUID
    ]

    func redact(_ text: String) -> String {
        var redacted = text
        for pattern in patterns {
            let range = NSRange(redacted.startIndex..., in: redacted)
            redacted = pattern.stringByReplacingMatches(
                in: redacted,
                range: range,
                withTemplate: "[REDACTED]"
            )
        }
        return redacted
    }
}

```

6.4 Log Destinations

New Relic Destination (Primary)

```

import NewRelic

struct NewRelicDestination: LogDestination {
    init?() {
        // Test Swift 6 compatibility
        guard Self.isCompatible() else {
            logError("New Relic SDK not compatible with Swift 6, falling back to backend")
            return nil
        }

        NewRelic.start(withApplicationToken: "YOUR_TOKEN")
    }

    private static func isCompatible() -> Bool {
        // Runtime compatibility check
        // This will be determined during Phase 2 implementation
        return true // Placeholder
    }

    func log(_ event: LogEvent) async {
        // Individual events for critical/error
        if event.level == .critical || event.level == .error {
            NewRelic.recordError(
                NSError(domain: event.category.rawValue, code: 0),
                attributes: event.toAttributes()
            )
        }
    }
}

```

```

func flush(_ events: [LogEvent]) async {
    for event in events {
        NewRelic.recordCustomEvent(
            "AppLog",
            attributes: event.toAttributes()
        )
    }
}

extension LogEvent {
    func toAttributes() -> [String: Any] {
        var attrs: [String: Any] = [
            "timestamp": timestamp.ISO8601Format(),
            "level": level.rawValue,
            "category": category.rawValue,
            "message": message
        ]

        for (key, value) in metadata {
            attrs[key] = value
        }

        if let duration = duration {
            attrs["duration"] = duration
        }
        if let memory = memoryUsage {
            attrs["memoryUsage"] = memory
        }

        return attrs
    }
}

```

Backend Destination (Fallback)

```

actor BackendDestination: LogDestination {
    private let apiClient: APIClient

    init(apiClient: APIClient = .shared) {
        self.apiClient = apiClient
    }

    func log(_ event: LogEvent) async {
        // No-op for individual events
    }

    func flush(_ events: [LogEvent]) async {
        do {
            let fingerprint = try await DeviceFingerprintService.shared.getFingerprint()

            try await apiClient.request(
                .post,
                path: "/logs/batch",
                body: LogBatchRequest(
                    deviceId: fingerprint,
                    events: events
                )
            ) as EmptyResponse
        }
    }
}

```

```

    } catch {
        os_log(.error, "Failed to send logs to backend: \(error)")
    }
}

}

struct LogBatchRequest: Codable, Sendable {
    let deviceId: String
    let events: [LogEvent]
}

struct EmptyResponse: Codable, Sendable {}

```

6.5 Performance Signposts

```

extension Logger {
    /// Begin performance signpost (returns immediately)
    nonisolated func signpostBegin(_ name: String, metadata: [String: String] = [:]) -> UUID {
        let id = UUID()

        Task.detached(priority: .utility) {
            let event = LogEvent(
                timestamp: Date(),
                level: .debug,
                category: .performance,
                message: "Signpost Begin: \(name)",
                metadata: metadata,
                signpost: SignpostInfo(name: name, id: id, state: .begin),
                duration: nil,
                memoryUsage: ProcessInfo.processInfo.physicalMemory
            )

            await self.buffer.append(event)

            // Also use os_signpost for Instruments
            os_signpost(.begin, log: .pointsOfInterest, name: name)
        }

        return id
    }

    /// End performance signpost (fire-and-forget)
    nonisolated func signpostEnd(_ name: String, id: UUID, metadata: [String: String] = [:]) {
        Task.detached(priority: .utility) {
            os_signpost(.end, log: .pointsOfInterest, name: name)

            let event = LogEvent(
                timestamp: Date(),
                level: .debug,
                category: .performance,
                message: "Signpost End: \(name)",
                metadata: metadata,
                signpost: SignpostInfo(name: name, id: id, state: .end),
                duration: nil,
                memoryUsage: ProcessInfo.processInfo.physicalMemory
            )

            await self.buffer.append(event)
        }
    }
}

```

```

}

// Usage (non-blocking):
let id = Logger.shared.signpostBegin("FetchLists")
// ... do work ...
Logger.shared.signpostEnd("FetchLists", id: id)
// Continues immediately, logging happens in background

```

6.6 Crash Reporting (Sentry)

```

import Sentry

// In AppDelegate
func application(
    _ application: UIApplication,
    didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey : Any]? = nil
) -> Bool {
    SentrySDK.start { options in
        options.dsn = "YOUR_SENTRY_DSN"
        options.environment = AppConfiguration.environment
        options.tracesSampleRate = AppConfiguration.environment == "debug" ? 1.0 : 0.1

        options.beforeSend = { event in
            // Attach user context
            if let user = await AuthenticationService.shared.currentUser {
                event.user = Sentry.User(userId: user.id)
            }
            return event
        }
    }

    return true
}

```

Sentry Free Tier:

- 5,000 events/month
- Crash reporting
- Performance monitoring
- Release health tracking
- 30-day retention

7. Models & Data Structures

All models are `Sendable` structs for thread-safe passing across concurrency boundaries.

```

struct GiftList: Identifiable, Codable, Sendable, Hashable {
    let id: String
    let name: String
    let description: String?
    let ownerId: String
    let ownerName: String
    let members: [ListMember]
    let createdAt: Date
    let updatedAt: Date

    var isOwner: Bool {

```

```

        // Determined based on current user
    }

    var isCoOwner: Bool {
        // Determined based on current user
    }
}

struct ListMember: Identifiable, Codable, Sendable, Hashable {
    let id: String
    let userId: String
    let userName: String
    let role: MemberRole
    let invitationStatus: InvitationStatus
}

enum MemberRole: String, Codable, Sendable {
    case owner
    case coOwner
    case member
    case viewer
}

enum InvitationStatus: String, Codable, Sendable {
    case pending
    case accepted
    case declined
}

struct GiftItem: Identifiable, Codable, Sendable, Hashable {
    let id: String
    let listId: String
    let name: String
    let description: String?
    let url: String?
    let imageUrl: String?
    let price: Decimal?
    let quantity: Int
    let isPurchased: Bool
    let isPending: Bool
    let isOffList: Bool
    let purchasedBy: String?
    let purchasedAt: Date?
    let createdBy: String
    let createdAt: Date
    let updatedAt: Date
}

struct User: Identifiable, Codable, Sendable, Hashable {
    let id: String
    let email: String
    let name: String
    let avatarUrl: String?
    let createdAt: Date
}

struct ListStats: Codable, Sendable, Hashable {
    let listId: String
    let totalItems: Int
    let purchasedItems: Int
    let totalBudget: Decimal
    let spentBudget: Decimal
}

```

```
    let remainingBudget: Decimal
}
```

8. Share Extension

8.1 Share Extension Architecture

The share extension allows users to add gift ideas directly from Safari or other apps.

```
// Share Extension Target
class ShareViewController: UIViewController {
    private let authService: AuthenticationService
    private let shareRepository: ShareRepository

    override func viewDidLoad() {
        super.viewDidLoad()

        // Extract shared content
        Task {
            await handleSharedContent()
        }
    }

    private func handleSharedContent() async {
        guard let item = extensionContext?.inputItems.first as? NSExtensionItem,
              let attachments = item.attachments else {
            return
        }

        for attachment in attachments {
            if attachment.hasItemConformingToTypeIdentifier(UTType.url.identifier) {
                if let url = try? await attachment.loadItem(
                    forTypeIdentifier: UTType.url.identifier
                ) as? URL {
                    await extractAndPresentMetadata(from: url)
                }
            }
        }
    }

    private func extractAndPresentMetadata(from url: URL) async {
        // Attempt iOS metadata extraction (5 second timeout)
        do {
            let metadata = try await withTimeout(seconds: 5) {
                try await URLMetadataExtractor.extract(from: url)
            }

            presentShareForm(url: url, metadata: metadata)
        } catch {
            // Fallback: send URL only, backend will extract
            presentShareForm(url: url, metadata: nil)
        }
    }

    private func presentShareForm(url: URL, metadata: URLMetadata?) {
        let formView = ShareFormView(
            url: url,
            metadata: metadata,
            onSave: { [weak self] item in
```

```

        await self?.saveItem(item)
    }
)

// Present SwiftUI view
let hostingController = UIHostingController(rootView: formView)
present(hostingController, animated: true)
}

}

// Share form with two-step list selection
struct ShareFormView: View {
    let url: URL
    let metadata: URLMetadata?
    let onSave: (ShareItemRequest) async -> Void

    @State private var mode: ShareMode = .myLists
    @State private var selectedList: GiftList?
    @State private var lists: [GiftList] = []
    @State private var quantity: Int = 1
    @State private var price: String = ""

    var body: some View {
        NavigationStack {
            Form {
                Section("What are you adding?") {
                    Picker("Mode", selection: $mode) {
                        Text("My List (something I want)").tag(ShareMode.myLists)
                        Text("Someone's List (a gift)").tag(ShareMode.memberLists)
                    }
                    .pickerStyle(.segmented)
                }

                Section("Select List") {
                    Picker("List", selection: $selectedList) {
                        ForEach(lists) { list in
                            Text(list.name).tag(list as GiftList?)
                        }
                    }
                }

                Section("Item Details") {
                    TextField("Name", text: .constant(metadata?.title ?? ""))
                    TextField("Price", text: $price)
                        .keyboardType(.decimalPad)
                    Stepper("Quantity: \(quantity)", value: $quantity, in: 1...99)
                }
            }
            .navigationTitle("Add to Christmas Tracker")
            .toolbar {
                ToolbarItem(placement: .cancellationAction) {
                    Button("Cancel") { /* dismiss */ }
                }
                ToolbarItem(placement: .primaryAction) {
                    Button("Save Now") {
                        Task {
                            await saveNow()
                        }
                    }
                }
                ToolbarItem(placement: .primaryAction) {
                    Button("Save as Pending") {
                        Task {

```

```

        await savePending()
    }
}

}

}
.task {
    await fetchLists()
}
}

}

private fun fetchLists() async {
    let type: ListType = mode == .myLists ? .owned : .member
    lists = try await ShareRepository.shared.fetchLists(type: type)
}

private fun saveNow() async {
    guard let listId = selectedList?.id else { return }

    let request = ShareItemRequest(
        url: url.absoluteString,
        title: metadata?.title,
        description: metadata?.description,
        imageUrl: metadata?.imageUrl,
        price: Decimal(string: price),
        quantity: quantity,
        listId: listId,
        isPending: false
    )

    await onSave(request)
}

private fun savePending() async {
    guard let listId = selectedList?.id else { return }

    let request = ShareItemRequest(
        url: url.absoluteString,
        title: metadata?.title,
        description: nil,
        imageUrl: nil,
        price: nil,
        quantity: 1,
        listId: listId,
        isPending: true
    )

    await onSave(request)
}

enum ShareMode {
    case myLists // Lists I own/co-own
    case memberLists // Lists I'm a member of
}

```

8.2 Shared Keychain Group

```

// Enable keychain sharing between app and extension
// Capabilities:

```

```

// - App Groups: group.com.christmastracker.shared
// - Keychain Groups: com.christmastracker

// KeychainService updated to support app groups
actor KeychainService {
    static let shared = KeychainService()

    private let appGroup = "group.com.christmastracker.shared"

    func set(_ value: String, for key: String, accessControl: AccessControl = .afterFirstUnlock)
throws {
        var query: [String: Any] = [
            kSecClass as String: kSecClassGenericPassword,
            kSecAttrAccount as String: key,
            kSecValueData as String: value.data(using: .utf8)!,
            kSecAttrAccessGroup as String: appGroup // Share with extension
        ]

        // ... rest of implementation
    }
}

```

8.3 URL Metadata Extraction

```

actor URLMetadataExtractor {
    static func extract(from url: URL, timeout: TimeInterval = 5) async throws -> URLMetadata {
        // Use LPMetadataProvider for Open Graph data
        let provider = LPMetadataProvider()

        let metadata = try await withTimeout(seconds: timeout) {
            try await provider.startFetchingMetadata(for: url)
        }

        return URLMetadata(
            title: metadata.title,
            description: metadata.summary,
            imageUrl: metadata.imageProvider?.url?.absoluteString,
            price: extractPrice(from: metadata.url?.absoluteString)
        )
    }

    private static func extractPrice(from urlString: String?) -> Decimal? {
        // Attempt to extract price from URL or content
        // Pattern matching for common price formats
        guard let urlString = urlString else { return nil }

        let patterns = [
            #"\"$([0-9]+\.[0-9]{2})"#,
            #"price=([0-9]+\.[0-9]{2})"#,
        ]

        for pattern in patterns {
            if let regex = try? NSRegularExpression(pattern: pattern),
                let match = regex.firstMatch(in: urlString, range: NSRange(urlString.startIndex...,
in: urlString)),
                let range = Range(match.range(at: 1), in: urlString) {
                let priceString = String(urlString[range])
                return Decimal(string: priceString)
            }
        }
    }
}

```

```

        return nil
    }
}

struct URLMetadata: Sendable {
    let title: String?
    let description: String?
    let imageUrl: String?
    let price: Decimal?
}

// Timeout helper
func withTimeout<T>(seconds: TimeInterval, operation: @escaping () async throws -> T) async throws
-> T {
    try await withThrowingTaskGroup(of: T.self) { group in
        group.addTask {
            try await operation()
        }

        group.addTask {
            try await Task.sleep(for: .seconds(seconds))
            throw TimeoutError()
        }

        let result = try await group.next()!
        group.cancelAll()
        return result
    }
}

struct TimeoutError: Error {}

```

9.3 Dark Mode Toggle

Users can control app appearance independently of system settings via Profile/Settings.

```

// Core/Configuration/AppearanceManager.swift

import SwiftUI

@Observable
class AppearanceManager {
    enum AppearanceMode: String, Codable, CaseIterable {
        case light = "Light"
        case dark = "Dark"
        case system = "System"
    }

    var currentMode: AppearanceMode {
        didSet {
            UserDefaults.standard.set(currentMode.rawValue, forKey: "appearanceMode")
            applyAppearance()
        }
    }

    init() {
        let savedMode = UserDefaults.standard.string(forKey: "appearanceMode")
        self.currentMode = AppearanceMode(rawValue: savedMode ?? "") ?? .system
    }
}

```

```

func applyAppearance() {
    Task { @MainActor in
        guard let scene = UIApplication.shared.connectedScenes.first as? UIWindowScene,
              let window = scene.windows.first else {
            return
        }

        switch currentMode {
        case .light:
            window.overrideUserInterfaceStyle = .light
        case .dark:
            window.overrideUserInterfaceStyle = .dark
        case .system:
            window.overrideUserInterfaceStyle = .unspecified
        }
    }
}

// Environment key
private struct AppearanceManagerKey: EnvironmentKey {
    static let defaultValue = AppearanceManager()
}

extension EnvironmentValues {
    var appearanceManager: AppearanceManager {
        get { self[AppearanceManagerKey.self] }
        set { self[AppearanceManagerKey.self] = newValue }
    }
}

// Usage in Profile/Settings
struct AppearanceSettingsView: View {
    @Environment(\.appearanceManager) private var appearanceManager

    var body: some View {
        VStack(alignment: .leading, spacing: AppSpacing.md) {
            Text("Appearance")
                .font(.appHeadline)
                .foregroundColor(.appTextPrimary)

            Picker("Theme", selection: $appearanceManager.currentMode) {
                ForEach(AppearanceManager.AppearanceMode.allCases, id: \.self) { mode in
                    Text(mode.rawValue).tag(mode)
                }
            }
                .pickerStyle(.segmented)
        }
        .padding(EdgeInsets.appCardPadding)
    }
}

```

10. Push Notifications

9.1 Notification Architecture

```

actor NotificationService {
    static let shared = NotificationService()
}

```

```

private let repository: NotificationRepository
private var deviceToken: String?

func requestAuthorization() async throws -> Bool {
    let center = UNUserNotificationCenter.current()

    let granted = try await center.requestAuthorization(options: [.alert, .sound, .badge])

    if granted {
        await MainActor.run {
            UIApplication.shared.registerForRemoteNotifications()
        }
    }

    return granted
}

func registerDevice(token: String) async throws {
    self.deviceToken = token

    let fingerprint = try await DeviceFingerprintService.shared.getFingerprint()

    let request = RegisterDeviceRequest(
        deviceToken: token,
        deviceFingerprint: fingerprint,
        platform: "ios",
        preferences: NotificationPreferences.default
    )

    try await repository.registerDevice(request)

    await Logger.shared.log(.info, "Device registered for notifications", category: .business)
}

func updatePreferences(_ preferences: NotificationPreferences) async throws {
    guard let token = deviceToken else {
        throw NotificationError.noDeviceToken
    }

    try await repository.updatePreferences(
        deviceToken: token,
        preferences: preferences
    )
}

func handleNotification(_ notification: UNNotification) async {
    let userInfo = notification.request.content.userInfo

    // Extract notification type and data
    guard let type = userInfo["type"] as? String else { return }

    await Logger.shared.log(
        .info,
        "Received notification: \(type)",
        category: .business,
        metadata: ["notificationId": notification.request.identifier]
    )

    // Handle deep linking
    if let deepLink = userInfo["deepLink"] as? String {
        await DeepLinkHandler.shared.handle(deepLink)
    }
}

```

```

    }
}

struct NotificationPreferences: Codable, Sendable {
    var itemAddedToCoOwnedList: Bool
    var watchedItemPurchased: Bool
    var budgetLimitApproaching: Bool
    var timeBasedReminders: Bool
    var newListCreated: Bool
    var itemAddedToMemberList: Bool

    static let `default` = NotificationPreferences(
        itemAddedToCoOwnedList: true,
        watchedItemPurchased: true,
        budgetLimitApproaching: true,
        timeBasedReminders: true,
        newListCreated: true,
        itemAddedToMemberList: true
    )
}

struct RegisterDeviceRequest: Codable, Sendable {
    let deviceToken: String
    let deviceFingerprint: String
    let platform: String
    let preferences: NotificationPreferences
}

```

9.2 Deep Linking

```

actor DeepLinkHandler {
    static let shared = DeepLinkHandler()

    func handle(_ urlString: String) async {
        guard let url = URL(string: urlString) else { return }

        // Parse deep link
        // Format: christmastracker://list/{listId}/item/{itemId}
        // Or Universal Link: https://christmastracker.com/list/{listId}/item/{itemId}

        let components = URLComponents(url: url, resolvingAgainstBaseURL: false)
        let pathComponents = components?.path.split(separator: "/").map(String.init) ?? []

        guard pathComponents.count >= 2 else { return }

        switch pathComponents[0] {
        case "list":
            let listId = pathComponents[1]
            if pathComponents.count >= 4 && pathComponents[2] == "item" {
                let itemId = pathComponents[3]
                await navigateToItem(listId: listId, itemId: itemId)
            } else {
                await navigateToList(listId: listId)
            }
        default:
            break
        }
    }

    private func navigateToList(listId: String) async {

```

```

        await MainActor.run {
            // Post notification to trigger navigation
            NotificationCenter.default.post(
                name: .navigateToList,
                object: listId
            )
        }
    }

    private func navigateToItem(listId: String, itemId: String) async {
        await MainActor.run {
            NotificationCenter.default.post(
                name: .navigateToItem,
                object: ["listId": listId, "itemId": itemId]
            )
        }
    }
}

extension Notification.Name {
    static let navigateToList = Notification.Name("navigateToList")
    static let navigateToItem = Notification.Name("navigateToItem")
}

```

9.3 Universal Links (Preferred)

```

// Associated Domains entitlement:
// applinks:christmastracker.com

// In SceneDelegate
func scene(_ scene: UIScene, continue userActivity: NSUserActivity) {
    guard userActivity.activityType == NSUserActivityTypeBrowsingWeb,
        let url = userActivity.webpageURL else {
        return
    }

    Task {
        await DeepLinkHandler.shared.handle(url.absoluteString)
    }
}

// Backend serves apple-app-site-association file:
// https://christmastracker.com/.well-known/apple-app-site-association
{
    "applinks": {
        "apps": [],
        "details": [
            {
                "appID": "TEAM_ID.com.christmastracker.ios",
                "paths": [
                    "/list/*",
                    "/item/*"
                ]
            }
        ]
    }
}

```

10. Testing Strategy

10.1 Unit Testing with Swift Testing

```
import Testing
@testable import ChristmasTracker

@Suite("ListService Tests")
struct ListServiceTests {

    @Test("Fetch lists successfully")
    func testFetchListsSuccess() async throws {
        // Arrange
        let mockRepository = MockListRepository()
        mockRepository.mockLists = [
            GiftList(id: "1", name: "Test List", /* ... */)
        ]

        let service = ListService(
            repository: mockRepository,
            datastore: ListDataStore()
        )

        // Act
        await service.fetchLists()

        // Assert
        #expect(service.lists.count == 1)
        #expect(service.lists[0].name == "Test List")
        #expect(service.isLoading == false)
    }

    @Test("Fetch lists handles error")
    func testFetchListsError() async throws {
        // Arrange
        let mockRepository = MockListRepository()
        mockRepository.shouldThrowError = APIError.httpError(statusCode: 500, data: Data())

        let service = ListService(
            repository: mockRepository,
            datastore: ListDataStore()
        )

        // Act
        await service.fetchLists()

        // Assert
        #expect(service.lists.isEmpty)
        #expect(service.error != nil)
        #expect(service.isLoading == false)
    }
}
```

10.2 UI Testing with XCUITest

```
import XCTest

final class LoginFlowUITests: XCTestCase {
```

```

var app: XCUIApplication!

override func setUpWithError() throws {
    continueAfterFailure = false
    app = XCUIApplication()
    app.launchArguments = ["--uitesting"]
    app.launch()
}

func testLoginSuccess() throws {
    // Enter credentials
    let emailField = app.textFields["Email"]
    emailField.tap()
    emailField.typeText("test@example.com")

    let passwordField = app.secureTextFields["Password"]
    passwordField.tap()
    passwordField.typeText("password123")

    // Tap login button
    app.buttons["Log In"].tap()

    // Verify navigation to dashboard
    XCTAssertTrue(app.navigationBars["Dashboard"].waitForExistence(timeout: 5))
}

func testLoginWithInvalidCredentials() throws {
    // Enter invalid credentials
    let emailField = app.textFields["Email"]
    emailField.tap()
    emailField.typeText("invalid@example.com")

    let passwordField = app.secureTextFields["Password"]
    passwordField.tap()
    passwordField.typeText("wrong")

    // Tap login button
    app.buttons["Log In"].tap()

    // Verify error message
    XCTAssertTrue(app.staticTexts["Invalid email or password"].waitForExistence(timeout: 5))
}
}

```

10.3 Coverage Goals

- **Unit Tests:** 80% coverage for business logic (Services, Repositories, DataStores)
- **UI Tests:** Critical user flows (login, create list, add item, purchase)
- **Integration Tests:** Network layer with mock server
- **Performance Tests:** List rendering, image loading, cache performance

11. CI/CD Pipeline (GitHub Actions)

11.1 Workflow Configuration

```

# .github/workflows/ios.yml
name: iOS CI

on:

```

```

push:
  branches: [ main, release/* ]
pull_request:
  branches: [ main ]

jobs:
  test:
    name: Run Tests
    runs-on: macos-14

    steps:
      - uses: actions/checkout@v4

      - name: Select Xcode
        run: sudo xcode-select -s /Applications/Xcode_26.0.app

      - name: Build and Test
        run: |
          xcodebuild test \
            -scheme ChristmasTracker \
            -destination 'platform=iOS Simulator,name=iPhone 15 Pro,OS=26.0' \
            -enableCodeCoverage YES \
            | xcpretty

      - name: Upload Coverage
        uses: codecov/codecov-action@v3
        with:
          files: ./coverage.lcov
          fail_ci_if_error: true

  build:
    name: Build Archive
    runs-on: macos-14
    needs: test
    if: github.ref == 'refs/heads/main'

    steps:
      - uses: actions/checkout@v4

      - name: Select Xcode
        run: sudo xcode-select -s /Applications/Xcode_26.0.app

      - name: Build Archive
        run: |
          xcodebuild archive \
            -scheme ChristmasTracker \
            -archivePath ./build/ChristmasTracker.xcarchive \
            -configuration Release

      - name: Upload Archive
        uses: actions/upload-artifact@v3
        with:
          name: ChristmasTracker.xcarchive
          path: ./build/ChristmasTracker.xcarchive

```

11.2 CI/CD Rules

- **Run tests on all PRs** before merge
- **Fail build on test failure** (no merge allowed)
- **Run tests on main/release branches** after merge
- **Generate code coverage reports** (target 80%)

- **Archive builds on main branch** for distribution
-

12. Backend API Requirements

12.1 Authentication Endpoints (New/Modified)

```
// NEW: Biometric authentication
POST /auth/biometric
Body: {
  refreshToken: string,
  deviceFingerprint: string
}
Response: {
  accessToken: string,
  refreshToken: string, // NEW token, old invalidated
  user: User
}

// MODIFIED: Login includes device fingerprint
POST /auth/login
Body: {
  email: string,
  password: string,
  deviceFingerprint: string
}
Response: {
  accessToken: string,
  refreshToken: string, // Tied to deviceFingerprint
  user: User
}

// MODIFIED: Refresh includes device fingerprint
POST /auth/refresh
Body: {
  refreshToken: string,
  deviceFingerprint: string
}
Response: {
  accessToken: string,
  refreshToken: string, // NEW token, old invalidated
}
```

Backend Changes:

- Add `deviceFingerprint` field to `RefreshToken` model
- Associate tokens with device fingerprints
- Validate fingerprint matches on token use
- Invalidate old token when issuing new one
- Limit to 10 devices per user
- Track `lastPasswordLogin` timestamp on `User` model
- Expire refresh tokens after 30 days if no password login

12.2 Share Extension Endpoints (New)

```
// NEW: Lightweight list fetch for share extension
GET /share/lists?type=owned|member
Response: {
```

```

    lists: Array<{
      id: string,
      name: string,
      ownerName: string
    }>
  }

// NEW: Add item via share extension
POST /share/item
Body: {
  url: string,
  title?: string,
  description?: string,
  imageUrl?: string,
  price?: number,
  quantity: number,
  listId: string,
  isPending: boolean
}
Response: {
  itemId: string
}

```

Backend Changes:

- Add `isPending` boolean to Item model
- Filter pending items from list queries (privacy)
- Pending items only visible to creator until finalized

12.3 Notification Endpoints (New)

```

// NEW: Register device for push notifications
POST /notifications/register
Body: {
  deviceToken: string,
  deviceFingerprint: string,
  platform: 'ios' | 'web',
  preferences: NotificationPreferences
}

// NEW: Update notification preferences
PUT /notifications/preferences
Body: {
  deviceToken: string,
  preferences: NotificationPreferences
}

// NEW: Unregister device
DELETE /notifications/unregister
Body: {
  deviceToken: string
}

```

Backend Changes:

- Add `DeviceNotificationSettings` table with per-device preferences
- Integrate with APNs (Apple Push Notification service)
- Schedule notifications based on user preferences
- Deep link format: `christmastracker://list/{listId}/item/{itemId}`

12.4 Logging Endpoint (New - Only if New Relic Incompatible)

```
// NEW: Batch log upload (fallback if New Relic SDK incompatible)
POST /logs/batch
Body: {
  deviceId: string,
  events: Array<LogEvent>
}
Response: 200 OK

// Backend forwards to New Relic
app.post('/logs/batch', async (req, res) => {
  const { deviceId, events } = req.body;

  for (const event of events) {
    newrelic.recordCustomEvent('iOSLog', {
      deviceId,
      ...event
    });
  }

  res.status(200).send();
});
```

13. Security Considerations

13.1 Data Protection

- **Keychain:** All sensitive data (tokens, device fingerprint UUID) stored in Keychain
- **Biometric Protection:** Refresh tokens protected by Face ID/Touch ID
- **PII Filtering:** Automatic redaction of PII in logs
- **HTTPS Only:** All network requests over TLS 1.3+
- **Certificate Pinning:** Consider for production (Phase 11)

13.2 Authentication Security

- **Token Rotation:** Refresh tokens rotate on every use
- **Device Binding:** Tokens tied to device fingerprints
- **Max Devices:** Limit 10 devices per user
- **Re-authentication:** Force password re-entry every 30 days
- **Session Timeout:** 2-hour access token lifetime

13.3 Code Security

- **Swift 6 Concurrency:** Data race prevention at compile time
- **No @unchecked Sendable:** Maintain strict type safety
- **Input Validation:** Zod validation on all API requests
- **Error Handling:** No sensitive data in error messages
- **Obfuscation:** Consider code obfuscation for production

14. Performance Considerations

14.1 Optimization Strategies

- **List Virtualization:** Use LazyVStack/LazyVGrid for large lists
- **Image Caching:** Use AsyncImage with custom cache

- **Network Caching:** 15-minute global cache reduces API calls
- **Background Operations:** All network/data operations in actors
- **Memory Management:** Clear caches on memory warnings

14.2 Performance Targets

- **App Launch:** <2 seconds cold start
 - **Screen Transitions:** <300ms navigation
 - **Network Requests:** <1 second typical response
 - **List Scrolling:** 60fps on iPhone 11+
 - **Memory Usage:** <150MB typical, <250MB peak
-

15. Accessibility

15.1 Requirements

- **VoiceOver:** Full support for screen readers
- **Dynamic Type:** Support all system text sizes
- **Contrast:** WCAG AA minimum (4.5:1 for normal text)
- **Touch Targets:** Minimum 44x44 points
- **Reduce Motion:** Respect reduced motion preference

15.2 Testing

- **VoiceOver Testing:** Manual testing with VoiceOver enabled
 - **Dynamic Type Testing:** Test at largest/smallest sizes
 - **Accessibility Inspector:** Xcode tool for automated checks
 - **Color Contrast:** Use tools to verify contrast ratios
-

16. Future Considerations

16.1 Foundation Models (Post-MVP)

- **Smart Gift Recommendations:** Use on-device AI for personalized suggestions
- **Natural Language Search:** "Find gifts under \$50 for Dad"
- **Price Predictions:** Estimate gift costs based on descriptions

16.2 Liquid Glass Design (Evaluate Later)

- **Postponed:** Current consensus is Liquid Glass may not persist
- **Monitor:** Track adoption and stability before implementing
- **Fallback:** Standard iOS design patterns work well

16.3 Additional Features

- **Offline Mode:** Add SwiftData for full offline support
 - **Watch App:** Extend to watchOS
 - **iPad Optimization:** Split view, drag-and-drop
 - **Widgets:** Home screen widgets for quick access
 - **Siri Shortcuts:** Voice commands for common actions
-

17. Deployment Strategy

17.1 Phased Rollout

1. **Alpha (Internal):** Development team testing

2. **Beta (TestFlight):** 50-100 family/friends
3. **Limited Release:** 1,000 users, monitor metrics
4. **General Availability:** App Store public release

17.2 Release Schedule

- **Phase 1-4:** Weeks 1-3 (Foundation, Network, Auth, Navigation)
- **Phase 5-8:** Weeks 4-8 (Lists, Items, Stats, Profile) - **MVP Cutline**
- **Phase 9-10:** Weeks 9-11 (Share Extension, Notifications)
- **Phase 11-12:** Weeks 12-13 (Polish, Testing, CI/CD)
- **Beta Release:** Week 14
- **General Availability:** Week 16

18. Glossary

- **Actor:** Swift concurrency primitive for thread-safe mutable state
- **Sendable:** Protocol marking types safe to pass across concurrency boundaries
- **@MainActor:** Attribute ensuring code runs on main thread
- **@Observable:** Macro enabling SwiftUI observation without ObservableObject
- **Repository:** Pattern abstracting data access from business logic
- **DataStore:** In-memory cache for domain data
- **Service:** Business logic coordinator between UI and data layers
- **xcconfig:** Xcode configuration file for build settings
- **Signpost:** Performance tracking marker for Instruments

19. References

- [Swift 6 Concurrency](#)
- [SwiftUI Observation Framework](#)
- [iOS 26 What's New](#)
- [Human Interface Guidelines](#)
- [App Store Review Guidelines](#)

20. Document History

Version	Date	Author	Changes
1.0	2026-02-03	Development Team	Initial architecture decision record

End of Architecture Decision Record